Draw It or Lose It

CS 230 Project Software Design Template

Version 3.0

Table of Contents

CS 230 Project Software Design Template 1

Table of Contents 2
Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
System Architecture View 4
Domain Model 4
Evaluation 6
Recommendations 9

Document Revision History

Version	Date	Author	Comments
1.0	03/16/2026	Israel Vasquez	Initial draft of the software design document for The Gaming Room web-based game application
2.0	04/02/2026	Israel Vasquez	Updated Evaluation section including server-side, client-side, and development tools analysis for Linux, Mac, Windows, and mobile platforms
3.0	04/15/2026	Israel Vasquez	Revised Recommendations section to improve objectivity and expanded system architecture, memory, storage, distributed systems, and security analysis

Executive Summary

The Gaming Room wants to expand its existing Android game, Draw It or Lose It, into a web-based application that can support multiple platforms. The main challenge is designing software that can work in a distributed environment while still enforcing the game rules and data requirements. In particular, the application must support multiple games, allow each game to contain one or more teams, allow each team to contain multiple players, ensure that game and team names are unique, and ensure that only one instance of the game service exists in memory at a time.

My proposed solution is to use an object-oriented Java design built around a service layer that manages all game data. This design uses a singleton pattern for the GameService class so that only one service instance exists in memory. It also uses an iterator pattern to search existing collections before adding new games, teams, or players, which helps enforce unique naming requirements. A shared base class called Entity should be used to hold common fields such as id and name, allowing Game, Team, and Player to inherit common behavior and reduce duplicate code.

This approach gives The Gaming Room a strong starting point for a scalable web-based solution. It keeps the design organized, supports future expansion, and provides a clear way to manage application data consistently across platforms. The client should understand that software design decisions made now, such as service structure, naming rules, and object relationships, will directly affect future deployment, storage, security, and hardware planning.

Requirements

The Gaming Room's business requirement is to expand Draw It or Lose It from a single-platform Android application into a web-based game that can be accessed from multiple platforms. The game must allow multiple teams to participate, and each team must support multiple players. The system must also make it easy for users to choose names by ensuring that game names and team names are unique.

The technical requirements are closely tied to these business needs. The application must manage games, teams, and players as distinct objects with unique identifiers. Only one instance of the game service can exist in memory at a time, which makes the singleton pattern an appropriate solution. The software must also be able to search through existing data efficiently to check whether names are already in use before creating new objects. Because the game is being developed as a web-based distributed application, the design should support future growth, portability, and communication across different operating systems and devices.

Design Constraints

One major design constraint is that the application must run in a web-based distributed environment rather than on a single local device. This means the software must be designed with portability and cross-platform support in mind. Instead of depending on device-specific behavior, the system should separate core game logic from the user interface so that multiple client types can interact with the same application services.

Another important constraint is the uniqueness requirement for game and team names. The system cannot simply create objects without checking existing records first. This requires controlled object creation and consistent searching through stored collections. The implication for development is that

methods such as addGame(), addTeam(), and addPlayer() must include logic to validate names before creating new instances.

A third constraint is that only one instance of the game service can exist in memory at one time. This limits how the service layer can be instantiated and makes the singleton pattern necessary. The implication is that all game data must be managed through one shared service object so that IDs remain consistent and duplicate services do not create conflicting records.

The application is also constrained by the need to support one-to-many relationships. A game contains multiple teams, and a team contains multiple players. Because of this, the class design must represent these relationships clearly and manage collections of child objects efficiently. This affects how the domain model is structured and how methods are written to add, search, and return related data.

Finally, the system must be maintainable and expandable. The client's current request is focused on core gameplay structure, but future versions may require networking, storage, user accounts, and browser-based interfaces. For that reason, the design should be modular, use inheritance where appropriate, and follow standard object-oriented practices so later changes can be added without rewriting the whole system.

System Architecture View

At this stage of development, the focus is on the software design and domain model rather than a full physical architecture. However, the intended system architecture is a web-based distributed application in which clients connect to a central application service over a network. In a future production environment, the client side would likely be a web browser running on desktop or mobile devices, while the server side would host the game logic, data services, and persistent storage.

This architecture supports the client's goal of expanding the game to multiple platforms because users would access the same centralized application through standard network communication rather than through separate platform-specific game logic. A layered design with presentation, application, and data tiers would improve maintainability and scalability.

Domain Model

The domain model for the Draw It or Lose It application is represented by a UML class diagram consisting of the classes Entity, Game, Team, Player, and GameService. These classes work together to model the structure and relationships required by the client's game application.

At the core of the design is the Entity class, which serves as a base class containing the shared attributes id and name. This allows all primary objects in the system to inherit these common properties, reducing code duplication and improving maintainability. The classes Game, Team, and Player all extend the Entity class, demonstrating the object-oriented principle of inheritance.

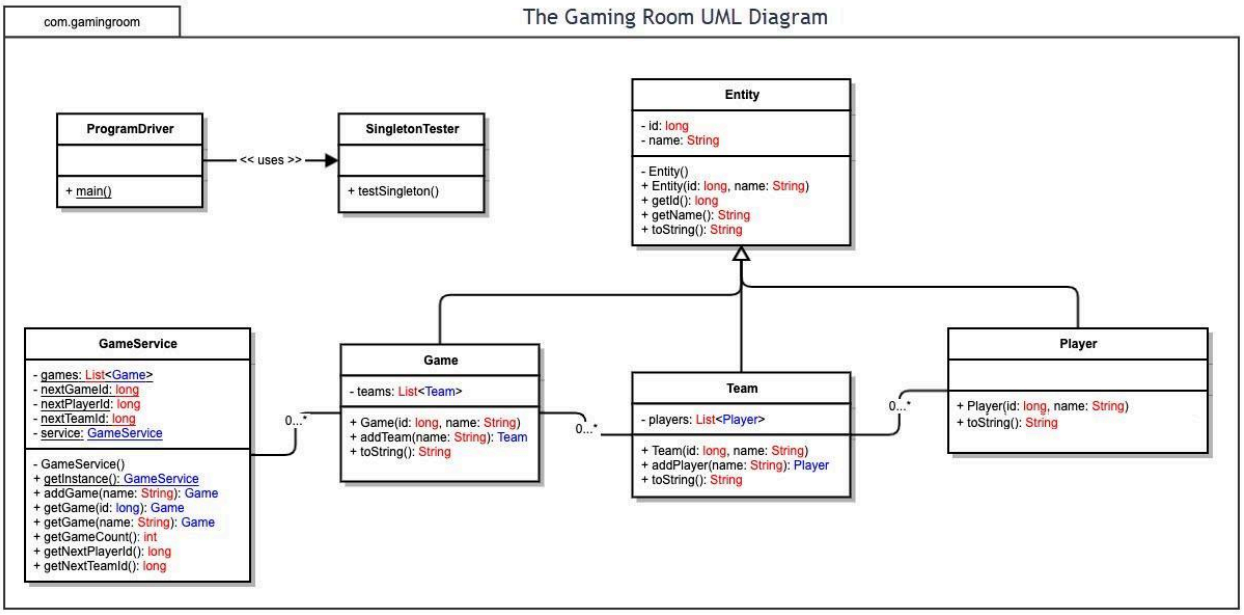
The relationships between the classes follow a hierarchical structure. A Game object contains a collection of Team objects, and each Team object contains a collection of Player objects. This reflects the client's requirement that a game can have one or more teams, and each team can have multiple players. These relationships demonstrate association and composition, as larger objects manage and contain related smaller objects.

The GameService class acts as the central service layer responsible for managing all game instances. It is also responsible for generating unique identifiers for games, teams, and players through dedicated methods, ensuring consistent ID management across the application. It maintains a collection of Game objects and is responsible for creating and retrieving games, teams, and players. This class implements the singleton design pattern, ensuring that only one instance of the service exists in memory at any given time. This satisfies the requirement that only one instance of the game service can exist and prevents inconsistencies in data management.

To enforce uniqueness of game names, team names, and player names, the design uses the iterator pattern to search existing collections before creating new objects. Methods such as addGame(), addTeam(), and addPlayer() iterate through their respective lists to check for existing names before adding new instances. This ensures that duplicate names are not created and that users can reliably check whether a name is already in use.

The design also demonstrates encapsulation, as each class manages its own data through controlled methods rather than exposing internal structures directly. This protects the integrity of the application's data and ensures that all validation rules are consistently applied.

Overall, the domain model effectively uses object-oriented principles and design patterns to create a structured, scalable, and maintainable system. By combining inheritance, encapsulation, association, and design patterns such as singleton and iterator, the model fulfills the client's requirements while supporting future expansion of the application.



Evaluation

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	Mac can be used for development and small-scale hosting, but it is not typically the strongest choice for production server deployment. It offers a stable Unix-based environment and supports modern development tools, which makes it useful during design, testing, and prototyping. However, Apple hardware is more expensive and less commonly used in enterprise server environments, which increases deployment cost and limits scalability. For a distributed web-based game that must support thousands of concurrent users, Mac is better suited for development rather than production hosting.	Linux is the strongest server-side option for this application. It is widely used for web hosting, cloud infrastructure, and distributed systems due to its stability, performance, and security. Linux is also cost-effective because most distributions are open source and do not require licensing fees. It supports Java, web servers (such as Apache or NGINX), containerization (Docker), and scalable cloud deployment environments. These capabilities make Linux ideal for handling high concurrency and scaling to thousands of players efficiently. Overall, Linux provides the best balance of performance, flexibility, and cost for this application.	Windows can host a web-based application successfully, especially in organizations that already rely on Microsoft technologies such as .NET or Azure. It provides a familiar administrative interface and strong enterprise-level support. However, Windows Server typically requires licensing fees and may consume more system resources compared to Linux. While it is a viable option, it is generally less cost-effective and less flexible for large-scale distributed web applications. For this project, Windows is a workable but less optimal solution compared to Linux.	Mobile devices are not appropriate for server-side hosting of this application. They lack the processing power, storage capacity, uptime reliability, and scalability required to support a distributed multiplayer system. Instead, mobile devices should function strictly as client endpoints that communicate with a centralized server infrastructure.

Client Side	Supporting Mac clients primarily involves ensuring that the web-based interface functions correctly across supported browsers such as Safari, Chrome, and Firefox. Because the application is delivered through the browser, there is no need to develop a separate native macOS application, which significantly reduces development time and cost. However, cross-browser testing is still required to ensure consistent functionality and performance.	Linux clients can also be supported through the browser-based approach. This eliminates the need for platform-specific desktop applications and reduces overall development complexity. The main consideration is ensuring compatibility across different browsers and desktop environments. This approach allows the application to reach Linux users without additional development overhead.	Windows clients represent a large portion of the user base, making them a critical consideration. A browser-based solution ensures compatibility without requiring a dedicated Windows application. Development efforts can focus on usability, performance, and browser compatibility rather than platform-specific builds. This approach is both cost-effective and efficient for supporting a broad user base.	Mobile devices are especially important because many users will access the game from smartphones and tablets. Using responsive web design allows the application to adapt to different screen sizes and touch-based interactions without requiring separate native applications for iOS and Android. This significantly reduces development time, cost, and maintenance effort. However, mobile support introduces additional challenges such as network variability, performance optimization, and user interface adjustments. Despite these challenges, a responsive web application provides the most practical and scalable solution for initial deployment.

Development Tools	Mac supports a wide range of development tools including Java, JDK, IntelliJ IDEA, Eclipse, and Visual Studio Code. Its Unix-based environment makes it well suited for backend development, scripting, and working with modern development workflows. Mac is also required for tools like Xcode if native iOS development is needed in the future. While development on Mac is powerful and flexible, the higher cost of Apple hardware can increase overall development expenses.	Linux provides the most complete and flexible environment for development and deployment. It supports Java, JDK, IntelliJ IDEA, Eclipse, VS Code, Git, Maven or Gradle, and is commonly used with web servers, containers (Docker), and databases. Linux is ideal for backend development and production environments, allowing developers to closely match deployment conditions. Because Linux is open source, there are little to no licensing costs, making it the most cost-effective platform for development teams.	Windows supports all major development tools such as Java, IntelliJ IDEA, Eclipse, VS Code, Git, and browser debugging tools. It is widely accessible and commonly used in enterprise environments, which can make onboarding developers easier. However, some development workflows, especially those involving servers, containers, or scripting, may require additional configuration compared to Linux. Windows may also involve licensing costs depending on the edition used, which can increase overall project cost.	For mobile support in a web-based application, development focuses on HTML, CSS, JavaScript, and responsive design frameworks. Tools include browser developer tools, testing platforms, and frameworks such as React for building responsive interfaces. This approach allows a single codebase to support both iOS and Android devices, reducing development time and cost. If native mobile applications are developed in the future, Android Studio (for Android) and Xcode (for iOS) would be required, which may introduce additional platform-specific expertise and potential team specialization.

Recommendations

1. **Operating Platform**: A Linux-based server platform is recommended for hosting the Draw It or Lose It application, paired with a browser-based client accessible on Windows, Mac, Linux, and mobile devices. Linux is widely used in distributed web environments due to its stability, scalability, and compatibility with modern development tools and cloud infrastructure. Additionally, its open-source nature can reduce licensing costs. A browser-based client allows users across multiple platforms to access the application without requiring platform-specific installations, which simplifies development and maintenance.

2. **Operating Systems Architectures**: The Linux operating system utilizes a multiuser, multitasking architecture with strong support for networking and process management. Its architecture is based on a monolithic kernel that efficiently manages system resources such as CPU scheduling, memory allocation, and device control. For this application, a layered system architecture can be implemented, separating the presentation layer (client interface), application layer (game logic and services), and data layer (storage). This separation improves maintainability and allows each layer to scale independently as system demand increases.

3. **Storage Management**: A relational database management system (RDBMS), such as MySQL or PostgreSQL, is an appropriate storage solution for this application. These systems provide structured data storage, enforce data integrity through constraints, and support efficient querying for game-related data such as players, teams, and sessions. As the application grows, persistent storage becomes necessary to maintain game state and user data beyond runtime. Additionally, Linux-based systems integrate well with database servers, making deployment and management more efficient.

4. **Memory Management**: Memory management for the application is handled through both the Linux operating system and the Java Virtual Machine (JVM). Linux uses virtual memory techniques, including paging and segmentation, to efficiently allocate system memory across processes. The JVM provides automatic memory management through garbage collection, which reclaims unused objects and reduces the risk of memory leaks. The use of a singleton pattern for the GameService ensures controlled memory usage by preventing duplicate service instances. Together, these techniques support stable performance and efficient resource utilization.

5. **Distributed Systems and Networks**: To support communication across multiple platforms, the application should follow a client-server distributed architecture. Clients connect to the centralized server over standard network protocols such as HTTP or HTTPS. This approach ensures that all users interact with a single source of game logic and data, maintaining consistency across sessions. The system must account for network dependencies such as latency, connectivity interruptions, and server availability. To address these challenges, the design can incorporate strategies such as load balancing, fault tolerance, and session persistence to improve reliability and scalability in distributed environments.

6. **Security**: Security should be implemented across all layers of the application to protect user data and maintain system integrity. This includes authentication and authorization mechanisms to control access, as well as encryption protocols such as HTTPS to secure data in transit. Data stored in the system should also be protected through proper access controls and, where appropriate, encryption at rest. Linux provides a strong security foundation with features such as

user permissions, firewall configurations, and regular security updates. Additionally, secure coding practices such as input validation and protection against common vulnerabilities (e.g., SQL injection) are essential for maintaining a secure application across platforms.